

Fast Sector Decomposition from Black-Box Symanzik Polynomials

1 The example triangle

We consider the scalar one-loop triangle with one incoming off-shell momentum and two outgoing massless momenta,

$$p_0 = p_1 + p_2, \quad p_1^2 = p_2^2 = 0, \quad p_0^2 = s = m_0^2. \quad (1)$$

All three internal propagators are taken to have the same mass m . A convenient routing is

$$D_0 = \ell^2 - m^2 + i0, \quad (2)$$

$$D_1 = (\ell + p_1)^2 - m^2 + i0, \quad (3)$$

$$D_2 = (\ell - p_2)^2 - m^2 + i0. \quad (4)$$

The scalar integral is

$$C_0(s; m^2) = \int \frac{d^D \ell}{i\pi^{D/2}} \frac{1}{D_0 D_1 D_2}. \quad (5)$$

With Feynman parameters $x_i \geq 0$, the projective representation is

$$C_0(s; m^2) = -\Gamma(\alpha) \int_0^\infty dx_0 dx_1 dx_2 \delta(1 - x_0 - x_1 - x_2) \frac{\mathcal{U}^{3-D}}{\mathcal{F}^\alpha}, \quad \alpha = 3 - \frac{D}{2}. \quad (6)$$

For this routing,

$$\mathcal{U} = x_0 + x_1 + x_2, \quad (7)$$

and

$$\mathcal{F} = m^2 \mathcal{U}^2 - s x_1 x_2 - i0. \quad (8)$$

Solving the simplex constraint by

$$x_0 = 1 - x_1 - x_2 \quad (9)$$

gives

$$0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1 - x_1, \quad (10)$$

and therefore

$$\mathcal{U} = 1, \quad \mathcal{F} = m^2 - s x_1 x_2 - i0. \quad (11)$$

For $D = 4 - 2\epsilon$, one has $\alpha = 1 + \epsilon$, and

$$C_0(s; m^2) = -\Gamma(1 + \epsilon) \int_0^1 dx_1 \int_0^{1-x_1} dx_2 (m^2 - s x_1 x_2 - i0)^{-1-\epsilon}. \quad (12)$$

Throughout the rest of the note, sector integrands I_s do not include the global prefactor $-\Gamma(\alpha)$. Thus an exact decomposition has the form

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s(y_s). \quad (13)$$

1.1 Endpoint sectors

The primary endpoint decomposition splits the simplex into two sectors.

Sector S_1 , $x_1 \geq x_2$.

$$X_{S_1}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{t}{1+z}, \frac{tz}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (14)$$

The Jacobian is

$$J_{S_1}(t, z) = \frac{t}{(1+z)^2}. \quad (15)$$

The pulled-back second Symanzik polynomial is

$$\mathcal{F}_{S_1}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (16)$$

In the massless endpoint case $m = 0$,

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}. \quad (17)$$

The expected monomial is therefore

$$M_{S_1}(t, z) = t^2 z. \quad (18)$$

Sector S_2 , $x_2 \geq x_1$.

$$X_{S_2}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{tz}{1+z}, \frac{t}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (19)$$

The Jacobian is again

$$J_{S_2}(t, z) = \frac{t}{(1+z)^2}. \quad (20)$$

The pulled-back polynomial is

$$\mathcal{F}_{S_2}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (21)$$

In the massless endpoint case,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z. \quad (22)$$

With these definitions, the two-sector representation of the original triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}(t, z), \quad (23)$$

$$I_{S_a}(t, z) = \frac{t}{(1+z)^2} \mathcal{U}(X_{S_a}(t, z))^{3-D} \mathcal{F}(X_{S_a}(t, z))^{-\alpha}. \quad (24)$$

Equivalently, after inserting the explicit polynomial,

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \left(m^2 - s \frac{t^2 z}{(1+z)^2} - i0 \right)^{-\alpha}. \quad (25)$$

For $D = 4 - 2\epsilon$ and $m = 0$, this becomes

$$C_0(s; 0) = -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad (26)$$

where

$$G_{S_a}(t, z) = (-s - i0)^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (27)$$

2 Direct approach

In the direct approach, each sector map X_s is substituted explicitly into $\mathcal{U}$ and $\mathcal{F}$. If the pulled-back polynomial factorizes as

$$\mathcal{F}_s(y) = \mathcal{F}(X_s(y)) = M_s(y) \phi_s(y), \quad (28)$$

then the sector integrand is built as

$$I_s^{\text{dir}}(y) = J_s(y) \mathcal{U}(X_s(y))^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (29)$$

The corresponding contribution to the original scalar integral is always understood as

$$C_0 = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s^{\text{dir}}(y_s), \quad (30)$$

with the sector domains $\mathcal{C}_s$ specified explicitly below.

2.1 Endpoint sector S_1

For $m = 0$,

$$\mathcal{U}_{S_1}(t, z) = 1, \quad (31)$$

and

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2} = M_{S_1}(t, z) \phi_{S_1}(t, z), \quad (32)$$

with

$$M_{S_1}(t, z) = t^2 z, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (33)$$

Therefore

$$I_{S_1}^{\text{dir}}(t, z) = \frac{t}{(1+z)^2} (t^2 z)^{-\alpha} \left(\frac{-s - i0}{(1+z)^2} \right)^{-\alpha}. \quad (34)$$

For $D = 4 - 2\epsilon$, i.e. $\alpha = 1 + \epsilon$,

$$I_{S_1}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (35)$$

2.2 Endpoint sector S_2

By symmetry,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z, \quad (36)$$

and hence

$$I_{S_2}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (37)$$

The full massless triangle is therefore

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}^{\text{dir}}(t, z) \\ &= -2\Gamma(1+\epsilon) (-s - i0)^{-1-\epsilon} \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \end{aligned} \quad (38)$$

3 Indirect approach

The indirect approach keeps the Symanzik polynomials closed. We assume only black-box evaluators

$$\widehat{\mathcal{U}}(x_0, x_1, x_2), \quad \widehat{\mathcal{F}}(x_0, x_1, x_2), \quad (39)$$

together with their partial derivatives with respect to the original parameters x_i . In practice, these derivative jets can be computed efficiently with dual numbers.

For a sector map X_s , define the pullbacks

$$U_s(y) = \widehat{\mathcal{U}}(X_s(y)), \quad F_s(y) = \widehat{\mathcal{F}}(X_s(y)). \quad (40)$$

Derivatives with respect to sector variables are obtained by chain rules. For example,

$$\partial_a F_s(y) = \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y), \quad (41)$$

and

$$\begin{aligned} \partial_a \partial_b F_s(y) &= \sum_{i,j} \frac{\partial^2 \widehat{\mathcal{F}}}{\partial x_i \partial x_j}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y) \frac{\partial X_{s,j}}{\partial y_b}(y) \\ &\quad + \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial^2 X_{s,i}}{\partial y_a \partial y_b}(y). \end{aligned} \quad (42)$$

Higher derivatives are obtained similarly, or more directly by propagating dual-number jets through the known sector map and then through the black-box polynomial evaluator.

The object to construct is the regular coefficient ϕ_s defined by

$$F_s(y) = M_s(y) \phi_s(y), \quad (43)$$

where M_s is the expected monomial for that sector. Once ϕ_s is available, the sector integrand is

$$I_s^{\text{ind}}(y) = J_s(y) U_s(y)^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (44)$$

At interior points this is pointwise identical to direct substitution, since

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)} \quad (45)$$

there. For the endpoint decomposition before any monomial extraction, the black-box form of the exact triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{3-D} \widehat{\mathcal{F}}(X_{S_a}(t, z))^{-\alpha}. \quad (46)$$

When $m = 0$, and hence $\widehat{\mathcal{F}}(X_{S_a}(t, z)) = t^2 z \phi_{S_a}(t, z)$, this same equality becomes

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1+2\epsilon} \\ &\quad \times (t^2 z)^{-1-\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \end{aligned} \quad (47)$$

3.1 Using subtraction terms

The subtraction construction is the one used to expose the Laurent poles in ϵ . At interior points one evaluates

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)}. \quad (48)$$

Boundary values are supplied by the continuous extension, using derivatives. For the first endpoint sector,

$$\phi_{S_1}(0, z) = \frac{1}{2z} [\partial_T^2 F_{S_1}(T, z)]_{T=0}, \quad (49)$$

$$\phi_{S_1}(t, 0) = \frac{1}{t^2} [\partial_Z F_{S_1}(t, Z)]_{Z=0}, \quad (50)$$

$$\phi_{S_1}(0, 0) = \frac{1}{2} [\partial_T^2 \partial_Z F_{S_1}(T, Z)]_{T=0, Z=0}. \quad (51)$$

Again, the derivatives of F_{S_1} are not symbolic derivatives of an opened polynomial. They are derivatives of $\widehat{\mathcal{F}} \circ X_{S_1}$, obtained from the black-box derivative jets and the chain rule.

For $D = 4 - 2\epsilon$, write each factorized endpoint integrand as

$$I_{S_a}^{\text{ind}}(t, z) = t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad a = 1, 2, \quad (52)$$

where

$$G_{S_a}(t, z) = \frac{1}{(1+z)^2} U_{S_a}(t, z)^{-1+2\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \quad (53)$$

The two-dimensional endpoint subtraction identity is

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G(t, z) \\ &= \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} [G(t, z) - G(0, z) - G(t, 0) + G(0, 0)] \\ & \quad - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G(0, z) - G(0, 0)] \\ & \quad - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G(t, 0) - G(0, 0)] + \frac{G(0, 0)}{2\epsilon^2}. \end{aligned} \quad (54)$$

Each bracket is less singular at the corresponding endpoint. This is the mechanism that turns endpoint singularities into explicit poles and finite integrals. Applied sector by sector, it gives the full massless triangle as

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \left\{ \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} [G_{S_a}(t, z) - G_{S_a}(0, z) \right. \\ & \quad \left. - G_{S_a}(t, 0) + G_{S_a}(0, 0)] \right. \\ & \quad - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G_{S_a}(0, z) - G_{S_a}(0, 0)] \\ & \quad \left. - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G_{S_a}(t, 0) - G_{S_a}(0, 0)] + \frac{G_{S_a}(0, 0)}{2\epsilon^2} \right\}. \end{aligned} \quad (55)$$

For the explicit massless triangle in sector S_1 ,

$$U_{S_1}(t, z) = 1, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (56)$$

Thus

$$G_{S_1}^{\text{exp}}(t, z) = (-s - i0)^{-1-\epsilon}(1+z)^{2\epsilon}. \quad (57)$$

This function is independent of t . Therefore

$$G_{S_1}^{\text{exp}}(t, z) - G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(t, 0) + G_{S_1}^{\text{exp}}(0, 0) = 0, \quad (58)$$

and

$$G_{S_1}^{\text{exp}}(t, 0) - G_{S_1}^{\text{exp}}(0, 0) = 0. \quad (59)$$

The only non-trivial subtraction piece is

$$G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(0, 0) = (-s - i0)^{-1-\epsilon} [(1+z)^{2\epsilon} - 1]. \quad (60)$$

The explicitly subtracted result for one endpoint sector is therefore

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz I_{S_1}^{\text{dir}}(t, z) \\ &= -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2}. \end{aligned} \quad (61)$$

The second sector gives the same contribution, so the full massless triangle is

$$C_0(s; 0) = -2\Gamma(1+\epsilon) \left\{ -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2} \right\}. \quad (62)$$

This is still a compact distributional expression. The object implemented in FSD is the Laurent expansion of this expression. Equivalently, the endpoint weights are expanded as plus distributions,

$$y^{-1-a\epsilon} = -\frac{1}{a\epsilon} \delta(y) + \sum_{n=0}^{\infty} \frac{(-a\epsilon)^n}{n!} \left[\frac{\log^n y}{y} \right]_+, \quad a > 0, \quad (63)$$

or, after pairing with a smooth test function h ,

$$\int_0^1 dy y^{-1-a\epsilon} h(y) = -\frac{h(0)}{a\epsilon} + \sum_{n=0}^{\infty} \frac{(-a\epsilon)^n}{n!} \int_0^1 dy \frac{\log^n y}{y} [h(y) - h(0)]. \quad (64)$$

The subtraction formula above arranges precisely that the non-localized remainders have the required endpoint zeros, so the plus prescription is realized numerically as ordinary integrable functions with explicit logarithms. For the explicit S_1 remainder,

$$\begin{aligned} z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] &= \epsilon \frac{2 \log(1+z)}{z} \\ &+ \epsilon^2 \frac{2 \log^2(1+z) - 2 \log z \log(1+z)}{z} + \mathcal{O}(\epsilon^3), \end{aligned} \quad (65)$$

which is regular at $z = 0$. The same expansion is applied to the prefactor $(-s - i0)^{-1-\epsilon}$, to $\Gamma(1+\epsilon)$, and to the black-box coefficient G_{S_a} in the generic implementation. Thus FSD samples the coefficients of the Laurent series, not the unexpanded $z^{-1-\epsilon}$ distribution itself. This expression was obtained without requiring any symbolic expansion in an implementation: only X_{S_a} , J_{S_a} , the known monomial $t^2 z$, and black-box values and derivative limits of $\hat{\mathcal{U}}$ and $\hat{\mathcal{F}}$ are required in each endpoint sector.

4 General parametric structure

For a scalar L -loop integral with N propagators of powers ν_i , define

$$A = \sum_{i=1}^N \nu_i, \quad D = D_0 + D_\epsilon \epsilon. \quad (66)$$

With the usual projective Feynman-parameter normalization, the parametric integral has the schematic form

$$I(\nu) = \mathcal{P}(D, \nu) \frac{\Gamma(A - LD/2)}{\prod_i \Gamma(\nu_i)} \int_{x_i \geq 0} \left[\prod_{i=1}^N dx_i x_i^{\nu_i-1} \right] \delta\left(1 - \sum_i x_i\right) \\ \times \mathcal{U}(x)^{A-(L+1)D/2} \mathcal{F}(x)^{-(A-LD/2)}. \quad (67)$$

The convention-dependent prefactor $\mathcal{P}(D, \nu)$ contains the signs, powers of $i\pi^{D/2}$, and any global normalization choices. It is not part of the sector-local black-box construction. What the sector processor needs are the affine powers

$$p_U(\epsilon) = A - (L+1)D/2, \quad (68)$$

$$p_F(\epsilon) = -(A - LD/2), \quad (69)$$

the parameter weights $\nu_i - 1$, and the retained black-box evaluators for $\mathcal{U}$ and $\mathcal{F}$.

After a sector map $x = X_s(y)$, the topology-specific sector generator should separate each possible endpoint monomial source,

$$J_s(y) \prod_i X_{s,i}(y)^{\nu_i-1} N_s(y) = J_{s,\text{reg}}(y) \prod_a y_a^{j_{s,a}}, \quad (70)$$

$$\mathcal{U}(X_s(y)) = M_{U,s}(y) \psi_s(y), \quad M_{U,s}(y) = \prod_a y_a^{u_{s,a}}, \quad (71)$$

$$\mathcal{F}(X_s(y)) = M_{F,s}(y) \phi_s(y), \quad M_{F,s}(y) = \prod_a y_a^{f_{s,a}}. \quad (72)$$

Here N_s denotes any smooth numerator or extra parameter-space factor. The regular functions ψ_s and ϕ_s are evaluated by black-box dual-number limits of $\mathcal{U}$ and $\mathcal{F}$; neither polynomial is symbolically opened inside the processor. The full endpoint power of y_a is then

$$\rho_{s,a}(\epsilon) = j_{s,a} + u_{s,a} p_U(\epsilon) + f_{s,a} p_F(\epsilon), \quad (73)$$

with any additional numerator or measure monomial powers included in $j_{s,a}$. This is the object that decides the subtraction problem. The logarithmic endpoint case is the simplest one,

$$\rho_{s,a}(\epsilon) = -1 + c_{s,a}\epsilon, \quad c_{s,a} \neq 0, \quad (74)$$

which produce explicit Laurent poles from $\int_0^1 dy_a y_a^{-1+c_{s,a}\epsilon}$. The generic interface is not restricted to this case: if $\rho_{s,a}(\epsilon) = \beta_{s,a} + c_{s,a}\epsilon$ with a negative integer $\beta_{s,a}$, the sector data must request the Taylor projector through order $-\beta_{s,a} - 1$.

For a uniform notation, let

$$P \in \{\mathcal{U}, \mathcal{F}\}, \quad r_{P,s,a} \in \mathbb{N}, \quad M_{P,s}(y) = \prod_{a \in A_s} y_a^{r_{P,s,a}}, \quad (75)$$

with $r_{\mathcal{U},s,a} = u_{s,a}$, $r_{\mathcal{F},s,a} = f_{s,a}$, and A_s the set of sector axes on which an endpoint monomial has been declared. Define

$$P_s(y) = \hat{P}(X_s(y)), \quad R_{P,s}(y) = \frac{P_s(y)}{M_{P,s}(y)}. \quad (76)$$

Thus $R_{\mathcal{U},s} = \psi_s$ and $R_{\mathcal{F},s} = \phi_s$. At an interior point this is just the quotient above. At a boundary point, let

$$B(y) = \{a \in A_s \mid y_a = 0\}, \quad \bar{B} = A_s \setminus B, \quad (77)$$

and introduce dual variables η_b for $b \in B$. The boundary jet is formed by

$$\hat{y}_a(\eta) = \begin{cases} \eta_a, & a \in B, \\ y_a, & a \notin B. \end{cases} \quad (78)$$

Then the regular residual is the Taylor coefficient

$$R_{P,s}(y) = \frac{[\prod_{b \in B} \eta_b^{r_{P,s,b}}] \hat{P}(X_s(\hat{y}(\eta)))}{\prod_{a \in \bar{B}} y_a^{r_{P,s,a}}}. \quad (79)$$

Here $[\eta^n]$ denotes the Taylor coefficient, not the derivative. In derivative notation the same statement is

$$R_{P,s}(y) = \frac{\left(\prod_{b \in B} \frac{1}{r_{P,s,b}!} \partial_{\eta_b}^{r_{P,s,b}} \right) \hat{P}(X_s(\hat{y}(\eta))) \Big|_{\eta=0}}{\prod_{a \in \bar{B}} y_a^{r_{P,s,a}}}. \quad (80)$$

The chain rule is applied at the level of truncated jets:

$$J_B^r(P_s)(y) = J_x^r(\hat{P})(X_s(y)) \circ J_B^r(X_s)(y), \quad (81)$$

where

$$\mathbf{r} = (r_{P,s,b})_{b \in B}. \quad (82)$$

Equivalently, for the first two derivative orders,

$$\partial_a P_s(y) = \sum_i \frac{\partial \hat{P}}{\partial x_i}(X_s(y)) \partial_a X_{s,i}(y), \quad (83)$$

$$\begin{aligned} \partial_a \partial_b P_s(y) &= \sum_{i,j} \frac{\partial^2 \hat{P}}{\partial x_i \partial x_j}(X_s(y)) \partial_a X_{s,i}(y) \partial_b X_{s,j}(y) \\ &\quad + \sum_i \frac{\partial \hat{P}}{\partial x_i}(X_s(y)) \partial_a \partial_b X_{s,i}(y). \end{aligned} \quad (84)$$

After extracting the declared monomials, every sector has the form

$$I_s(\epsilon) = \int_{[0,1]^d} d^d y \left[\prod_{a \in A_s} y_a^{\rho_{s,a}(\epsilon)} \right] G_s(y, \epsilon), \quad (85)$$

where

$$G_s(y, \epsilon) = J_{s,\text{reg}}(y) R_{\mathcal{U},s}(y)^{p_U(\epsilon)} R_{\mathcal{F},s}(y)^{p_F(\epsilon)} \quad (86)$$

times any remaining regular numerator factor. Write

$$\rho_{s,a}(\epsilon) = \beta_{s,a} + c_{s,a}\epsilon, \quad \beta_{s,a} \in \mathbb{Z}, \quad (87)$$

and define the endpoint Taylor order

$$N_{s,a} = \max(-\beta_{s,a} - 1, 0). \quad (88)$$

For one endpoint variable,

$$\begin{aligned} \int_0^1 dy y^{\beta+c\epsilon} G(y, \epsilon) &= \int_0^1 dy y^{\beta+c\epsilon} \left[G(y, \epsilon) - \sum_{n=0}^N \frac{y^n}{n!} \partial_y^n G(0, \epsilon) \right] \\ &\quad + \sum_{n=0}^N \frac{\partial_y^n G(0, \epsilon)}{n! (\beta + n + 1 + c\epsilon)}. \end{aligned} \quad (89)$$

The first line is finite at $y = 0$, while the second line contains the explicit ϵ -poles.

For several endpoint axes, introduce the Taylor projector

$$(\mathcal{T}_a G)(y, \epsilon) = \sum_{n=0}^{N_{s,a}} \frac{y_a^n}{n!} [\partial_{y_a}^n G(y, \epsilon)]_{y_a=0}, \quad \mathcal{R}_a = 1 - \mathcal{T}_a. \quad (90)$$

Then the recursive subtraction identity can be written compactly as

$$\begin{aligned} I_s(\epsilon) &= \sum_{B \subseteq A_s} \sum_{\{0 \leq n_b \leq N_{s,b}\}_{b \in B}} \left[\prod_{b \in B} \frac{1}{n_b! (\beta_{s,b} + n_b + 1 + c_{s,b}\epsilon)} \right] \\ &\quad \times \int dy_{\bar{B}} \left[\prod_{a \in A_s \setminus B} y_a^{\beta_{s,a} + c_{s,a}\epsilon} \right] \left[\prod_{a \in A_s \setminus B} \mathcal{R}_a \right] \left[\prod_{b \in B} \partial_{y_b}^{n_b} G_s(y, \epsilon) \right]_{y_B=0}. \end{aligned} \quad (91)$$

In the logarithmic case $\beta_{s,a} = -1$, one has $N_{s,a} = 0$, and the identity reduces to

$$\begin{aligned} I_s(\epsilon) &= \sum_{B \subseteq A_s} \left[\prod_{b \in B} \frac{1}{c_{s,b}\epsilon} \right] \int dy_{\bar{B}} \left[\prod_{a \in A_s \setminus B} y_a^{-1+c_{s,a}\epsilon} \right] \\ &\quad \times \left[\prod_{a \in A_s \setminus B} (1 - \mathcal{T}_a) \right] [G_s(y, \epsilon)]_{y_B=0}, \quad (\mathcal{T}_a G)(y, \epsilon) = G(y, \epsilon)|_{y_a=0}. \end{aligned} \quad (92)$$

Finally, expanding

$$G_s(y, \epsilon) = \sum_{k \geq 0} \epsilon^k G_s^{(k)}(y), \quad \prod_{a \in A_s \setminus B} y_a^{c_{s,a}\epsilon} = \sum_{r \geq 0} \frac{\epsilon^r}{r!} \left(\sum_{a \in A_s \setminus B} c_{s,a} \log y_a \right)^r, \quad (93)$$

gives the Laurent coefficients explicitly as ordinary finite integrals.

This audit leads to a useful interface boundary. A topology definition must retain the global parametric metadata: loop count, propagator powers, dimension, prefactor description, and affine powers of $\mathcal{U}$ and $\mathcal{F}$. A sector definition must retain the map, regular sector prefactor, monomial powers of $\mathcal{U}$ and $\mathcal{F}$, monomial powers from the Jacobian/measure or numerator, and the singular axes. The generic processor can then assemble $\rho_{s,a}$, evaluate ψ_s and ϕ_s from dualized black-box evaluators, and apply the appropriate localized subtraction formula.

The numerical implementation now follows this interface boundary directly. The Laurent accumulator is variable length: once sectors are known, the minimum Laurent order is set by the scalar bound $-2L$, where L is the loop count, and the requested maximum order is selected at run time. The sector endpoint pole depth is checked against this bound before integration. The subtraction engine is recursive for any number of endpoint axes with negative-integer base powers, using the Taylor order $N_{s,a} = -\beta_{s,a} - 1$. Logarithmic plus distributions are therefore the special case $N_{s,a} = 0$. Positive monomial powers factored by pySecDec but not belonging to singular axes are kept as regular multiplicative factors in G_s . Non-integer endpoint powers, unregulated endpoint powers, non-unit propagator powers, and tensor numerators are deliberately rejected in the current scalar implementation, because they require additional metadata or analytic input.

5 Outlook: Schwinger sampling with momentum-space numerators

The construction above assumes that the loop momenta have already been integrated out, so all non-trivial information enters through the Symanzik polynomials and possible parameter-space numerator factors. A complementary path is useful when the denominator part should still be organized in Feynman/Schwinger parameters, but the numerator is easier to keep as a momentum-space black box. Schematically, start from a regulated integral

$$I[f] = \int d^{DL}k \frac{f(k)}{\prod_{e \in E} (D_e(k) + i0)^{\nu_e}}. \quad (94)$$

Using Schwinger parameters, or equivalently the projective Feynman-parameter form of the same denominator product,

$$\prod_{e \in E} \frac{1}{(D_e(k) + i0)^{\nu_e}} = \frac{\Gamma(\nu)}{\prod_e \Gamma(\nu_e)} \int_{\mathbb{P}_+^{|E|-1}} \prod_e dx_e x_e^{\nu_e-1} \left[\sum_e x_e (D_e(k) + i0) \right]^{-\nu}, \quad \nu = \sum_e \nu_e, \quad (95)$$

one obtains an integral over positive edge parameters and loop momenta, without replacing $f(k)$ by tensor-reduction formulae. The quadratic denominator combination can be written schematically as

$$\sum_e x_e D_e(k) = (k - Q(x))^T A(x) (k - Q(x)) + \mathcal{V}(x), \quad \mathcal{V}(x) = \frac{\mathcal{F}(x)}{\mathcal{U}(x)}, \quad (96)$$

with the appropriate $+i0$ prescription or later contour deformation understood. Introducing an overall Schwinger scale and auxiliary momentum variables then gives a representation of the schematic form

$$I[f] \propto \int_{\mathbb{P}_+^{|E|-1}} \prod_e dx_e x_e^{\nu_e-1} \frac{1}{\mathcal{U}(x)^{D/2} \mathcal{V}(x)^\omega} \times \int_0^\infty d\lambda \lambda^{\omega-1} e^{-\lambda} \int d\mu(q) f(k(x, q, \lambda)), \quad (97)$$

where

$$\omega = \sum_e \nu_e - \frac{DL}{2}. \quad (98)$$

Here $d\mu(q)$ denotes a fixed reference measure for the auxiliary momentum variables after the quadratic form has been normalized. The important feature is not the particular choice of that reference measure. The important feature is that the numerator remains a callable function of momenta, while the Feynman-parameter part still exposes $\mathcal{U}$, $\mathcal{F}$, endpoint monomials, and sector maps to the same FSD machinery as before.

After sector decomposition of the parameter variables, the intended structure is

$$I[f] = \sum_s \int_{[0,1]^{d_s}} dy \int d\lambda d\mu(q) \prod_{a \in A_s} y_a^{\rho_{s,a}(\epsilon)} G_s(y, \lambda, q; \epsilon) f(k_s(y, \lambda, q)). \quad (99)$$

The subtraction operators act on the sector variables y_a . The auxiliary variables λ and q are spectators for the endpoint algebra:

$$\mathcal{S}_s [G_s(y, \lambda, q; \epsilon) f(k_s(y, \lambda, q))], \quad (100)$$

with $\mathcal{S}_s$ the same local Taylor-subtraction operator used for the ordinary parameter-space integrand. Thus the future topology object would store not only $\mathcal{U}$, $\mathcal{F}$, and the parametric

exponents, but also the momentum routing, the quadratic denominator data, and the map from sampled auxiliary variables to loop momenta. The integrand definition would expose a momentum-space numerator callback

$$f : k \mapsto f(k), \quad (101)$$

and FSD would provide

$$z \in [0, 1]^n \mapsto (K_{\text{FSD}}(z), k(z)), \quad (102)$$

so that the numerical integrand is $K_{\text{FSD}}(z)f(k(z))$.

This is conceptually different from momentum-space tropical sampling, such as the approach implemented in `momtrop`; see <https://github.com/alpha100p/momtrop>. The FSD version would deliberately use sector decomposition and subtraction for the Feynman-parameter part. It therefore inherits the larger sector count and will not compete with tropical sampling on the factorial-sector problem for finite parameter integrals. The advantage is different: the same framework can expose endpoint poles in ϵ , and, once contour deformation or another threshold treatment is added, it is not tied to a finite Euclidean integration domain. For generic higher-order endpoint subtractions one may also need derivatives of the composed numerator $f(k_s(y, \lambda, q))$, or an explicit regularity contract showing that only boundary values are required. This is a boundary condition for the future backend, not part of the current scalar implementation. Moreover, the momentum-space numerator must be understood as a D -dimensional numerator, not merely a four-dimensional one. Rational terms from the $(D - 4)$ -dimensional components have to be retained, for example through the usual μ -term bookkeeping

$$k_{[D]}^2 = k_{[4]}^2 + \mu^2, \quad \mu^2 \equiv k_{[D-4]}^2, \quad (103)$$

with signs and metric conventions fixed consistently with the chosen denominator prescription.

6 Implementation in FSD

The repository contains the modular implementation used for the numerical checks described above. It is referred to simply as FSD. The executable entry point is

$$\text{FSD.py}. \quad (104)$$

A local virtual environment is created at

$$\text{.venv}, \quad (105)$$

and `OneLoopBridge` is required as an external dependency. The benchmark values are obtained from the Python binding calls

$$\text{oneloop_bridge.three_point}(0,0,s,m2,m2,m2), \quad (106)$$

$$\text{oneloop_bridge.four_point}(0,0,0,0,s12,s23,m2,m2,m2,m2). \quad (107)$$

The only implementation described here is the modular FSD code.

6.1 Code organization

The implementation is organized so that the black-box rule is visible directly in the code layout:

- `FSD.py` is the command-line steering layer. It parses options, validates kinematics, builds the request object, prints summaries, calls the benchmark, launches the integration, and renders the final table.

- `sectors_generator.py` owns the declarative sector data: sector maps X_s , regular sector prefactors, endpoint monomials $M_{U,s}$ and $M_{F,s}$, monomial powers from Jacobian/measure or numerator factors, and singular axes. The lowering of these declarative expressions to Symbolica callbacks is an explicit preparation step.
- `integrand.py` owns the topology definition and the generic `SectorProcessor`. The topology retains $\mathcal{U}$ and $\mathcal{F}$ for display and evaluator construction, but the processor never substitutes a sector map into them symbolically.
- `subtraction_formula.py` owns the generated Symbolica endpoint-subtraction formulae. It provides both the full sector-signature formula backend and the newer lower-signature endpoint-projector backend.
- `integrator.py` owns the Havana sampling loop, manual Laurent coefficient accumulation, progress reporting, target-accuracy stopping, and hot-path timing profile.
- `benchmark.py`, `formatting.py`, `definitions.py`, and `utils.py` isolate `OneLoopBridge` access, colored output, shared data containers, and parenthesis uncertainty formatting.
- `dot_parser.py`, `kinematics.py`, `dot_topology.py`, and `pysecdec.bridge.py` implement the experimental GammaLoop-convention DOT-file path. The bridge is the only FSD-owned module that imports `pySecDec`. It uses `pySecDec` to construct $\mathcal{U}, \mathcal{F}$ and sector data, then converts them into the same `TopologyDefinition` and `SectorDefinition` objects consumed by the generic processor.

6.2 DOT-file path through pySecDec

The command-line option

```
--dot-file path/to/integral.dot
```

selects the GammaLoop DOT-backed topology path, together with

```
--kinematics path/to/kinematics.yaml
```

The YAML file supplies numeric symbol values and scalar-product replacement rules. The current schema is

```
values:
  s12: -1.0
  s23: -1.0
  mt: 0.0
replacements:
  p1*p1: 0
  p2*p2: 0
  p1*p2: s12/2
```

Values and replacement right-hand sides are evaluated with Symbolica, not with SymPy. DOT edges touching nodes with `style=invis` are external half-edges. An edge `ext -> v` is interpreted as incoming momentum, and `v -> ext` as outgoing momentum. Internal edge attributes `mass` are either numeric strings or symbols present in the YAML `values` block. These mass symbols are resolved to their numeric values before `pySecDec` generation, so the decomposition sees the actual massless or massive polynomial for the requested kinematic point. Flow directions are retained in the parsed DOT data; `pySecDec` receives bare external momentum symbols, with the sign convention represented by the scalar-product replacement rules.

The implemented generation sequence is

$$\text{DOT file} \longrightarrow \text{ParsedDotGraph}, \quad (108)$$

$$\text{ParsedDotGraph} + \text{KinematicsDefinition} \longrightarrow \text{pySecDec LoopIntegralFromGraph}, \quad (109)$$

$$\text{pySecDec LoopIntegralFromGraph} \longrightarrow \text{TopologyDefinition}, \quad (110)$$

$$\text{pySecDec sectors} \longrightarrow \{\text{SectorDefinition}\}. \quad (111)$$

The selected decomposition method is controlled by

```
--sector-method geometric
--sector-method geometric_ku
--sector-method iterative
```

The default is `iterative`, which does not require `Normaliz`. For geometric decomposition, `Normaliz` is required either from `pySecDecContrib` or through `--normaliz-executable`. If no `Normaliz` executable is visible, `FSD` rejects the geometric mode before generation starts and the user can select `geometric_ku` or `iterative` instead. The execution engine is controlled by

```
--dot-engine fsd
--dot-engine pysecddec
--dot-engine both
```

In `fsd` mode, `pySecDec` is used only for generation; integration is performed by `FSD/Havana` using black-box `Symbolica` evaluators. In `pysecddec` mode, `pySecDec` generates, compiles, loads, and runs its own C++ integration package. In `both` mode both paths are run for comparison.

Generation timings are recorded at detailed granularity and also grouped into three headline buckets:

$$T_{\mathcal{U}, \mathcal{F}} := T_{\text{DOT}} + T_{\text{kin}} + T_{\text{LoopIntegralFromGraph}} + T_{\text{U/F extraction}}, \quad (112)$$

$$T_{\text{sectors}} := T_{\text{pySecDec decomposition}} + T_{\text{SectorDefinition conversion}}, \quad (113)$$

$$T_{\text{Symbolica}} := T_{\text{scalar U/F evaluators}} + T_{\text{sector map/J evaluators}} + T_{\text{dual U/F evaluators}}. \quad (114)$$

The pre-integration summary prints these three buckets, $\mathcal{U}, \mathcal{F}$, evaluator parameter order, the dual-evaluator mode, a capped sector table showing at most twenty sectors, and statistics for total sector count, number of singular axes, U/F monomial power tuples, endpoint pole depth, maximum dimension, and maximum Laurent pole order.

The current DOT examples include the massless triangle and box, plus six massive Euclidean multi-loop topologies: a two-loop two-point kite, a three-loop two-point self-energy graph, two two-loop three-point graphs, and two three-loop three-point graphs. The larger three-point examples have six and eight internal lines, respectively. The triangle and box reproduce the hard-coded examples through the DOT path. The multi-loop examples are finite in the tested kinematics and are run with `pySecDec` contour deformation disabled. They exercise `pySecDec` U/F construction, sector conversion, retained `Symbolica` evaluators, regular monomial factors, and the same `FSD/Havana` integration loop without the package-generation cost of double-box ladder examples. DOT examples with global Gamma prefactor poles are not yet part of the validated set, because the current `pySecDec`-convention prefactor convolution assumes a prefactor regular at $\epsilon = 0$.

6.3 Prepared sectors and black-box processing

The code mirrors the split used in this note. Sectors are prepared independently of the Symanzik polynomials. Each sector stores only

$$\{X_s, J_{s,\text{reg}}, M_{U,s}, M_{F,s}, \text{measure/numerator monomial powers, singular axes}\}, \quad (115)$$

plus Symbolica evaluators generated from these known sector expressions. The processing layer receives these prepared sectors together with black-box Symbolica evaluators

$$\widehat{\mathcal{U}}(x; p), \quad \widehat{\mathcal{F}}(x; p), \quad (116)$$

where p denotes the external invariants and masses. After the evaluators are built, the processing code does not inspect or symbolically rewrite the expressions for $\mathcal{U}$ or $\mathcal{F}$.

At runtime the processor receives only a **SectorDefinition**, a **TopologyDefinition**, and numeric sample points. For a sector point y it evaluates

$$x = X_s(y), \quad \widehat{\mathcal{U}}(x), \quad \widehat{\mathcal{F}}(x), \quad J_s(y), \quad (117)$$

through prebuilt Symbolica evaluators. If the sector has no endpoint monomial, the contribution is evaluated directly. If a monomial is declared, the residuals

$$\psi_s(y) = \frac{\widehat{\mathcal{U}}(X_s(y))}{M_{U,s}(y)}, \quad \phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_{F,s}(y)} \quad (118)$$

are evaluated by these quotients in the interior. On endpoint samples the same quantities are recovered as Taylor coefficients of the composed black-box $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$. In the default backend these coefficients are obtained from dualized $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators. The dual jets of the known sector map provide the chain-rule input, so no symbolic expansion of $\mathcal{U}(X_s(y))$ or $\mathcal{F}(X_s(y))$ is performed. The scalar $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators are constructed once with the topology. The command line exposes the following Taylor-evaluator preparation policies:

- **--pregenerate-dual-evaluators**, the default, builds one dualized $\widehat{\mathcal{U}}$ evaluator and one dualized $\widehat{\mathcal{F}}$ evaluator for each unique sector Taylor shape before integration starts.
- **--lazy-dual-evaluators-generation** keeps the cache-on-first use behavior: the scalar evaluator is copied and dualized the first time a shape is requested. This time is reported separately as Taylor evaluator setup time, not as ordinary evaluator time.
- **--pregenerate-single-overall-dual-evaluator** builds an envelope dual evaluator. For an integration dimension d , let $r_d = \max_s |A_s|$ be the maximum number of singular axes among sectors of that dimension. Each sector multi-index $\alpha \in \mathbb{N}^{|A_s|}$ is embedded as

$$\iota_s(\alpha) = (\alpha_1, \dots, \alpha_{|A_s|}, 0, \dots, 0) \in \mathbb{N}^{r_d}. \quad (119)$$

The envelope shape is

$$\mathcal{J}_d^{\text{env}} = \bigcup_{s: \dim s = d} \{ \iota_s(\alpha) : \alpha \in \mathcal{J}_s \}. \quad (120)$$

After evaluation, the output columns indexed by $\iota_s(\alpha)$ are remapped back to the sector-native shape $\mathcal{J}_s$.

- **--symbolic-derivatives** does not dualize the topology-level $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators. Instead, it builds ordinary non-dual Symbolica evaluators for the symbolic partial derivatives

$$\partial_x^\rho \widehat{P}(x), \quad \widehat{P} \in \{ \widehat{\mathcal{U}}, \widehat{\mathcal{F}} \}, \quad (121)$$

with ρ selected from the polynomial support and the largest sector Taylor order requested. These derivative evaluators are shared by all sectors. For a sector map

$$X_s(y + \eta) = X_s(y) + H_s(y, \eta), \quad (122)$$

the sector Taylor coefficients are reconstructed numerically through

$$[\eta^\alpha] \widehat{P}(X_s(y + \eta)) = [\eta^\alpha] \sum_\rho \frac{\partial_x^\rho \widehat{P}(X_s(y))}{\rho!} H_s(y, \eta)^\rho. \quad (123)$$

This is an explicit chain-rule backend for cross-checking the dualized evaluator path; it is not expected to be faster in the current Python implementation.

In FSD DOT mode, `TopologyDefinition` and `SectorDefinition` objects are generated and prepared once in the master process. Multi-worker integration uses a fork context so workers inherit this prepared state and wrap it in a `SectorProcessor`. Workers do not call `pySecDec`, do not rerun sector generation, and do not create `Normaliz` temporary directories. If a fork context is unavailable, multi-worker DOT integration fails explicitly instead of silently regenerating sectors at runtime.

The generic processor then constructs

$$g_s(y; \epsilon) = J_{s,\text{reg}}(y) \psi_s(y)^{p_U(\epsilon)} \phi_s(y)^{p_F(\epsilon)}, \quad (124)$$

multiplied by any regular positive monomial factors from the sector metadata. It then assembles the endpoint powers $\rho_{s,a}(\epsilon)$ and applies the recursive localized Taylor-subtraction formula for all declared singular axes. Higher-rank endpoint structures therefore use the same code path as the one- and two-axis triangle/box examples. Unsupported endpoint structures, such as non-integer powers or powers without an epsilon regulator, raise an explicit diagnostic instead of being guessed.

6.4 Subtraction backends and lower-signature projector

The command-line option

```
--subtraction-backend recursive|formula|projector-formula
```

selects how the endpoint algebra is evaluated after the sector data and U/F coefficient evaluators have been prepared. The default backend is `projector-formula`; the other two paths are kept as diagnostics and cross-checks.

The `recursive` backend is the direct vectorized Python/Numpy implementation of Eq. (91). It obtains the regular Taylor–Laurent coefficients

$$g_{s,\alpha,r}^{(Z)}(y_Z) = [\epsilon^r \eta^\alpha] G_s(y_Z, \eta_Z; \epsilon) \quad (125)$$

from the black-box U/F/J Taylor evaluators, and then performs the inclusion–exclusion sum over endpoint projectors in Python.

The `formula` backend builds one Symbolica evaluator for the complete sector-subtraction expression. Its cache signature contains the sector axes, all U/F/Jacobian/measure monomial powers, the native Taylor-coefficient layout, the endpoint powers, and the requested Laurent range. This path is a useful correctness reference because the endpoint algebra is evaluated inside Symbolica, but the formula is close to sector-specific.

The `projector-formula` backend splits the expression into two layers. First, the already-existing black-box Taylor path computes the sector-specific regular coefficients $g_{s,\alpha,r}^{(Z)}$. Second, these coefficients are passed to a Symbolica evaluator that knows only the endpoint projector algebra,

$$\mathcal{P}_{\{\beta_a, c_a, N_a\}, k_{\min}:k_{\max}} : \left\{ y_a, g_{s,\alpha,r}^{(Z)} \right\} \longmapsto \{I_{s,k}(y)\}_{k=k_{\min}}^{k_{\max}}. \quad (126)$$

Its cache key is only

$$(n_{\text{axes}}, \{(\beta_a, c_a)\}_{a=1}^{n_{\text{axes}}}, \{N_a\}_{a=1}^{n_{\text{axes}}}, \{k_{\min}, \dots, k_{\max}\}), \quad (127)$$

not the sector map or the U/F polynomial data. Thus U and F remain black-box coefficient providers, while many sectors can share the same generated endpoint-projector evaluator.

For the DOT double-box test this reduces the generated subtraction expression family from 133 full sector formula signatures to 20 endpoint-projector signatures. For the stored triple-box metadata, the corresponding count is about 863 current full signatures versus 158 ordered

endpoint-projector signatures. The present **projector-formula** backend is therefore a lower-signature generated-formula path. It does not yet remove all Python cost, because the regular coefficients $g_{s,\alpha,r}^{(Z)}$ are still assembled by the shared Python/Numpy Taylor-series layer.

A further runtime accelerator prepares regular-Taylor coefficient formulae. These formulae still do not contain $\mathcal{U}$, $\mathcal{F}$, or any sector-map substitution. Their inputs are only sector coordinates and already computed Taylor coefficients of the mapped regular Jacobian, $\mathcal{U}$, and $\mathcal{F}$. The Symbolica evaluator then performs the residual monomial division, the $\mathcal{U}$ - and $\mathcal{F}$ -power algebra, the logarithms, and the regular epsilon expansion that produces $g_{s,\alpha,r}^{(Z)}$. In symbolic-derivative mode the evaluator is fed only the ancestor-closed Taylor shape required by the corresponding regular formula, rather than the full sector dual shape. When a selected sector has a small enough family of regular signatures, FSD can also pregenerate Symbolica chain-rule composition formulae for those minimized regular Taylor shapes. These formulae receive numerical original-parameter derivatives of $\mathcal{U}$ or $\mathcal{F}$ and numerical sector-map Taylor coefficients; they do not contain $\mathcal{U}$, $\mathcal{F}$, or X_s after substitution. In that mode the chain-rule reconstruction of the needed U/F jets is moved into Symbolica evaluators instead of being assembled in Python. This is currently enabled when the chain-rule formula request workload is moderate. The original sector-specific signature would expand to 101018 regular formulae for the full triple-box all-sector run. Lower residual-input signatures and sparse output requests make the prepared family much smaller, but some high-axis formulae remain expensive to build cold. The current default therefore prepares only regular formulae with at most five Taylor axes and Taylor-box volume at most 64, controlled by `--regular-taylor-formula-axis-limit` and `--regular-taylor-formula-volume-limit`. In the triple-box run this prepares 147 regular formulae, uses 6 curated regular-Taylor assets, skips 29 harder high-axis formulae, uses shipped direct endpoint-projector assets for 382 sectors across 55 endpoint signatures, and keeps all-sector generation near 5.6 seconds. A selected PSD62 diagnostic shows why the guard is needed: even with the sparse output signature that avoids the full Cartesian Taylor box, trying to build eight six-axis regular formulae took about 226 seconds. The resulting one-sample runtime was evaluator-dominated, but the guarded default avoids cold-building that formula family and keeps generation acceptable. The remaining sparse path now propagates the sparse endpoint output-pair request down to the black-box Taylor source shape and is much faster than the old dense rectangular fallback, but it is still not the desired production runtime for the hardest triple-box sectors. For a selected PSD649 leading-pole probe, pregenerated chain-rule formulae reduce the diagnostic from 124.6 seconds of Python time for two samples to 32.2 seconds for ten samples, with no precision rescue. The lowered chain-rule signature removes sector names and map-expression strings from the key; caching the resulting map-jet layout, reusing the structural map layout at runtime, and caching one-index ancestor closures reduce chain-rule formula preparation from about 6.3 seconds to about 3.8 seconds. A curated direct endpoint-projector asset for the same universal endpoint signature gives a much better selected-sector path: ten ordinary-precision samples take 0.368 seconds of Python time and 0.054 seconds of Symbolica evaluator time, with compatible central values. In the current threshold-54 default, selected 4000-sample, 10-worker probes of PSD814 and PSD855 use direct cached endpoint projectors and reach 6.7% relative MC error on the deepest pole without any precision rescue. These probes are stable, but still Python-heavy; the remaining issue for high-depth signatures is regular-source sparse series assembly and variance, not endpoint numerical instability. A direct chain-rule lookup cache now avoids recomputing the structural chain-rule signature after pregeneration, which gives a modest reduction in the short PSD814 profile. The regular fallback also reuses the same sparse convolution ladder for integer powers and logarithms, and evaluates all requested original-parameter derivatives through one Symbolica multi-output evaluator. A selected 4000-sample PSD814 profile now gives $\text{PythonT} = 41.78\text{ s}$, $\text{EvalT} = 16.45\text{ s}$, and 4.11×10^3 evaluator microseconds per sample per worker. This is still Python-heavy, but removes one repeated sparse-series construction and reduces per-derivative evaluator dispatch

in the guarded fallback. Forcing high-axis regular-Taylor formulae for the same sector was stopped after 180 seconds before integration, at moderate RSS, so the current default keeps direct endpoint projectors enabled but leaves unvetted high-axis regular-Taylor formulae behind the guard. A focused PSD1057 forced-formula probe gives the same conclusion for a different all-sector contributor: with 400 requested samples and one worker, it produced no result file after 301 seconds and plateaued near 11.4 GiB RSS before the watchdog timeout stopped it. The bottleneck in that mode is therefore cold construction of the oversized regular formula, not Monte Carlo sampling. For the full active triple-box sector set, the chain-rule request count exceeds the guard and is reported as a skipped cold-build request; this does not mean that sectors are missing. It only means that those auxiliary chain-rule evaluator signatures are left to the runtime fallback rather than turning all-sector generation into a long cold-build phase. For leading-pole probes, sectors whose endpoint pole depth is too shallow to overlap the requested Laurent range are now returned as exact zero before any endpoint-subtraction algebra is assembled. This removes guaranteed-zero shallow sectors from the hot path, but it does not by itself fuse the regular-source assembly feeding genuinely high-depth sectors. The Symbolica `functions=` evaluator hook was inspected as a possible way to delegate more of the sparse composition, but it supplies symbolic function definitions that are inlined into the evaluator; it is not a runtime Python callback mechanism. The useful directions are therefore smaller universal generated expressions, better Symbolica-side factorization, or a native sparse-series primitive.

6.5 IBP lowering and endpoint-projector cache

The optional command-line flag

```
--IBP_reduce_to_log_endpoint
```

is implemented for the `projector-formula` backend. Its purpose is to avoid evaluating endpoint projectors directly on powers such as $y^{-2+c\epsilon}$ or $y^{-3+c\epsilon}$. For one endpoint variable, the elementary integration-by-parts identity used by the implementation is

$$\int_0^1 dy y^{-n+c\epsilon} h(y) = \frac{h(1)}{c\epsilon - n + 1} - \frac{1}{c\epsilon - n + 1} \int_0^1 dy y^{-n+1+c\epsilon} h'(y), \quad n > 1. \quad (128)$$

Applying this repeatedly reduces the sampled endpoint power to a logarithmic endpoint $y^{-1+c\epsilon}$. In multiple endpoint variables, the code applies the same lowering independently on each axis and enumerates the resulting boundary and derivative terms. The extra derivatives are not a new kind of input: they are further Taylor coefficients of the same black-box regular function g_s , recovered from the already available dual or symbolic derivative evaluator path. Run-file settings can be overridden explicitly:

```
--no-ibp-reduce-to-log-endpoint
```

This is useful for selected-sector diagnostics because the non-IBP projector can have fewer child Taylor requests even when the all-sector non-IBP formula generation is too expensive.

The implementation is deliberately split into topology-independent endpoint blocks. An IBP-lowered sector is represented as

$$I_{s,k}^{\text{IBP}}(y) = \sum_{\tau \in \mathcal{T}_s} \sum_r b_{\tau,r} \mathcal{P}_{\tau,k-r}(y_\tau, \{\partial_y^{\alpha_\tau} g_{s,\beta,r'}(y_{\bar{B}_\tau}, 0_{B_\tau})\}), \quad (129)$$

where $\mathcal{T}_s$ is the finite set of IBP boundary/derivative terms, $b_{\tau,r}$ are the epsilon-series coefficients of the IBP rational prefactors, and $\mathcal{P}_\tau$ is a logarithmic endpoint-projector formula. The projectors $\mathcal{P}_\tau$ know nothing about the topology, U, F, masses, or sector-map variable names; they only know endpoint powers, Taylor orders, and the requested Laurent range.

These endpoint-projector and regular-Taylor formulae are cached in

```
assets/subtraction_formulae
```

The cache stores parseable Symbolica expression strings and metadata, not compiled evaluator objects. The validated endpoint-projector cache files and a small validated regular-Taylor subset are kept in the curated cache subdirectory and are treated as FSD source assets. They are therefore default-on and need not be regenerated in a fresh checkout. Regular-Taylor cache files are promoted much more selectively: they become default behavior only after the corresponding evaluator has shown a runtime benefit, because a cached expression only removes cold formula generation, not necessarily hot evaluator cost. The first curated regular-Taylor subset is the six small PSD213-class signatures, adding about 100 KiB. On a cache hit, generation only parses the cached expressions and rebuilds the Symbolica evaluators. The curated endpoint-projector subset currently contains 488 validated universal formulae, about 49 MB, and is treated as shipped FSD source data. For the current triple-box leading-pole smoke, all 1972 sectors and prepared formulae were generated in about 5.6 seconds: 0.18 seconds for U/F construction, 1.11 seconds for sector generation, and 4.29 seconds for Symbolica evaluator construction. The same legacy guarded run prepared 147 regular-Taylor formula signatures, left 29 harder high-axis regular signatures behind the default cold-build guard, and selected direct cached endpoint projectors for 382 sectors across 55 endpoint signatures. The prepared-bundle workflow is the stricter production path: it materializes all sectors and serialized evaluator artifacts under the output directory before integration. When IBP lowering would produce a large compound endpoint tree and a direct endpoint-projector formula is already curated for that exact universal signature, FSD uses the direct cached endpoint projector by default. The threshold is controlled by

```
--direct-projector-cache-term-threshold 54
```

and can be set to 0 to always prefer the IBP compound projector. This policy keeps the all-sector non-IBP cold-generation path disabled while letting validated direct endpoint projectors become ordinary FSD source assets. The shipped triple-box run preset now sets this threshold to 0 for the complete all-sector six-order run, because the faster selected-sector direct endpoint-projector path can spike close to or above a 30 GiB memory budget when all sectors are sampled on 10 workers. With `batch-size=1`, $\epsilon^{-6} \dots \epsilon^{-1}$, and a 30 GiB watchdog, the run was interrupted at 324 seconds wall time when RSS reached 30.19 GiB. The partial result contained 370 accumulated samples, used no precision-rescue evaluations, and produced

	ϵ^{-6}	ϵ^{-5}	ϵ^{-4}	ϵ^{-3}	ϵ^{-2}	ϵ^{-1}
FSD	0.145(154)	2.05(3.02)	-72.4(67.0)	-805(593)	-2159(4303)	-984(24956)

in the pySecDec display convention. This is a memory/runtime stress result, not a precision validation. At runtime, the IBP child terms share one regular Taylor envelope for each boundary/zero projector, so repeated child requests for smaller Taylor boxes reuse the largest already-required box. In the symbolic-derivative backend, the U and F Taylor reconstructions also share the same sector-map Taylor context before applying their separate black-box derivative evaluators. These optimizations are useful, but IBP still introduces additional boundary and derivative child projectors. In the legacy selected-sector comparison for PSD213, a 400-sample leading-pole run without IBP used 32 regular signatures and took about 0.88 seconds in Python plus 5.63 seconds in Symbolica evaluator calls. The corresponding IBP run used 108 regular signatures and took about 1.91 seconds in Python plus 7.38 seconds in Symbolica evaluator calls. The current sparse residual-input path for the same non-IBP sector uses 6 regular signatures; a 400-sample run spent about 0.34 seconds in Python and 4.24 seconds in Symbolica evaluator calls. The central value was unchanged within Monte Carlo precision, but the IBP path was slower in the legacy comparison. A current selected-sector PSD62 diagnostic reinforces this point. With the run preset's IBP lowering enabled, the sector uses many logarithmic child

projectors. With `--no-ibp-reduce-to-log-endpoint`, the same leading two coefficients are obtained from a direct higher-power endpoint projector. For 10000 ordinary-precision samples, the no-IBP control found $\epsilon^{-5} = -0.0734(70)$, i.e. a 9.52% relative Monte Carlo error, with no precision-rescue samples. In the current threshold-54 default, the depth-six representative sectors PSD814 and PSD855 use direct cached endpoint projectors. With 4000 samples on 10 workers they both reached $\epsilon^{-6} = -0.000680(45)$, a 6.7% relative MC error, without any precision-rescue sample. The ϵ^{-5} coefficient is sector-dependent and can be compatible with zero, so relative errors there are less diagnostic. These are selected-sector convergence checks, not an all-sector triple-box validation.

The numerical role of IBP lowering is stability and algebraic control, not a guaranteed raw per-sample speedup. The stability rescue itself comes from the full multiprecision path: near an endpoint, the sample, sector-map jets, U/F/J Taylor coefficients, projector inputs, and final projector evaluator call are all promoted to the requested Decimal precision and evaluated through Symbolica’s `evaluate_complex_with_prec` API before the final sector weight is cast back to double precision. In the PSD213 point probe with all singular coordinates set to 10^{-12} , ordinary double precision produced non-finite lower Laurent coefficients, 32 and 80 digits were still unstable for the deepest lower orders, and 160 and 300 digits agreed in the displayed coefficients. The default runtime policy is therefore deliberately conservative: promote endpoint distances below 10^{-3} to 32 digits, below 10^{-6} to 100 digits, and below 10^{-8} to 1000 digits for this endpoint-stability probe.

6.6 Havana sampling and stopping

FSD uses Symbolica’s Havana integrator as a sampler rather than as a single black-box scalar integrator. One discrete dimension chooses the sector and each sector has a continuous adaptive grid of the same dimension:

$$\text{discrete sector id} \quad \otimes \quad y \in [0, 1]^d. \quad (130)$$

Lower-support subtraction pieces are localized into the same highest-dimensional sample space by dummy unit-density dependence on unused variables. This avoids separate grids for bulk, edge, and corner pieces.

Each sampled point contributes to all Laurent coefficients through the same manual accumulator,

$$A_k = \frac{1}{N} \sum_{n=1}^N w_n I_{k,s_n}(y_n), \quad k \in \{k_{\min}, k_{\min} + 1, \dots, k_{\max}\}. \quad (131)$$

Here $k_{\min} = -2L$ by default, and the upper endpoint of the range is the CLI-selected maximum epsilon order. Havana is trained on the norm of the last requested Laurent coefficient, so the training observable follows the coefficient actually controlling the requested truncation. The adaptive grid is cloned at the beginning of an iteration; batch training samples are accumulated into the clone and merged back into the live grid only at the end of a completed iteration.

The target-accuracy stop is evaluated after each accumulated batch, not only after a full Havana iteration. Thus the final returned coefficients contain all completed batches up to the stopping point. In parallel mode the driver keeps only a worker-sized window of batches in flight, so early stopping does not submit the entire iteration unnecessarily. The option `--batch-size` therefore controls both vectorization size and mid-iteration stopping granularity.

6.7 Timing profile

The timing profile reported by the progress bar and final table is a work-time profile:

$$T_{\text{prof}} = T_{\text{Python}} + T_{\text{eval}} + T_{\text{Havana}}. \quad (132)$$

Here T_{eval} is the sum of Symbolica evaluator wall times over all workers, T_{Python} is NumPy and Python glue on workers plus master-side submission, result retrieval, accumulation, and progress rendering, and T_{Havana} is Havana sampling, training accumulation, grid merge, and grid update time. The displayed evaluator average is

$$\langle t_{\text{eval}} \rangle = \frac{10^6 T_{\text{eval}}}{N_{\text{accepted}}} \quad \text{microseconds per sample per worker,} \quad (133)$$

because T_{eval} is already summed over workers. Taylor evaluator setup is tracked separately as $T_{\text{Taylor gen}}$. For the default pregenerated dual mode and for the symbolic-derivative mode this belongs to the generation bucket $T_{\text{Symbolica}}$. For lazy mode it can occur during the first integration batches, but it is still reported separately and is not included in T_{eval} .

6.8 Implemented modes and conventions

For the triangle, the command-line interface supports massive and massless modes. In the validated finite massive region, endpoint monomial factorization is not applied. The implementation evaluates

$$C_0(s; m^2) = - \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz J_{S_a}(t, z) \hat{\mathcal{U}}(X_{S_a}(t, z))^{-1} \hat{\mathcal{F}}(X_{S_a}(t, z))^{-1}. \quad (134)$$

In massless Euclidean mode, the implementation uses the subtraction formula of Section 3.2 and returns the coefficients of

$$C_0(s; 0) = \frac{C_{-2}}{\epsilon^2} + \frac{C_{-1}}{\epsilon} + C_0 + \mathcal{O}(\epsilon). \quad (135)$$

The default output convention is the OneLOopBridge convention. In this convention the universal Gamma/Euler factors are stripped from the displayed Laurent coefficients. For massless endpoint representations this also amounts to applying the finite convention shift described in the direct construction. The option `--prefactor-convention` controls only the overall scalar-integral normalization shown in the output table. The default value `raw` displays the raw OneLOopBridge normalization. The value `feynman` multiplies both the FSD coefficients and the benchmark coefficients by

$$\text{TO_FEYNMAN} = -\frac{1}{16\pi^2}. \quad (136)$$

In either display convention, values with Monte Carlo uncertainty are printed with the two-significant-digit parenthesis notation.

6.9 Implemented box example

The implementation also contains a scalar box example with massless external legs,

$$D_0(0, 0, 0, 0, s_{12}, s_{23}; m^2). \quad (137)$$

For equal internal masses, the simplex polynomial used by the implementation is

$$\mathcal{U} = x_0 + x_1 + x_2 + x_3, \quad \mathcal{F} = m^2 \mathcal{U}^2 - s_{12} x_0 x_2 - s_{23} x_1 x_3 - i0. \quad (138)$$

After imposing $\mathcal{U} = 1$, the finite four-point integral in $D = 4$ is

$$D_0 = \sum_{b=0}^3 \int_0^1 dy_0 \int_0^1 dy_1 \int_0^1 dy_2 J_b(y) \hat{\mathcal{F}}(X_b(y))^{-2}. \quad (139)$$

Here b labels the primary sector in which x_b is the largest Feynman parameter. In sector b ,

$$x_b = \frac{1}{1 + y_0 + y_1 + y_2}, \quad x_{j \neq b} = \frac{y_j}{1 + y_0 + y_1 + y_2}, \quad (140)$$

with the three y_j 's assigned in increasing order to the non-dominant parameters. The Jacobian is

$$J_b(y) = \frac{1}{(1 + y_0 + y_1 + y_2)^4}. \quad (141)$$

For $m = 0$, each primary sector has the form

$$\mathcal{F}(X_b(y)) = \frac{A_b y_\ell + B_b y_r y_s}{(1 + y_0 + y_1 + y_2)^2}, \quad (142)$$

where one of the two bilinears $x_0 x_2$ and $x_1 x_3$ contains the dominant parameter and is therefore linear in the primary variables. The implementation does not need the symbolic values of A_b and B_b ; it only uses the known positions of the linear variable y_ℓ and product variables y_r, y_s . The three secondary sectors are

$$L : \quad y_r = uv, \quad y_\ell = u, \quad y_s = w, \quad M_L = u, \quad (143)$$

$$P : \quad y_r = u, \quad y_s = v, \quad y_\ell = uvw, \quad M_P = uv, \quad (144)$$

$$Q : \quad y_r = u, \quad y_\ell = uv, \quad y_s = vw, \quad M_Q = uv. \quad (145)$$

The secondary Jacobian equals the displayed monomial in each case, so the regular prefactor after monomial extraction is again

$$\frac{1}{(1 + y_0(u, v, w) + y_1(u, v, w) + y_2(u, v, w))^4}. \quad (146)$$

The residual coefficient

$$\phi_{b\alpha}(u, v, w) = \frac{\mathcal{F}(X_{b\alpha}(u, v, w))}{M_\alpha(u, v, w)} \quad (147)$$

is evaluated by the black-box $\mathcal{F}$ away from the endpoint and by dual-number Taylor coefficients at $u = 0$ and $v = 0$. The one-monomial sector L uses the one-variable subtraction formula, while the P and Q sectors use the two-variable subtraction formula.

The raw coefficients produced directly by the massless box sectors are converted to the OneLOopBridge stripped convention as

$$D_{-2} = A_{-2}, \quad D_{-1} = A_{-1} + A_{-2}, \quad D_0 = A_0 + A_{-1} + \frac{\pi^2}{6} A_{-2}. \quad (148)$$

The current massless box command validates Euclidean kinematics, $s_{12} < 0$ and $s_{23} < 0$. Non-Euclidean massless kinematics are outside the present implementation.

6.10 Example commands

The massive triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s 1.0 --m 1.0
```

The same check in the Feynman-normalized prefactor convention can be displayed with

```
.venv/bin/python FSD.py \
--integral triangle \
--s 1.0 \
--m 1.0 \
--prefactor-convention feynman
```

The massless Euclidean triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s -1.0 --m 0.0
```

The massive box example can be run with

```
.venv/bin/python FSD.py \  
--integral box \  
--s12 0.5 \  
--s23 0.7 \  
--m 1.0
```

The massless Euclidean box check can be run with

```
.venv/bin/python FSD.py \  
--integral box \  
--s12 -1.0 \  
--s23 -2.0 \  
--m 0.0
```

The same triangle and box can be routed through DOT input and pySecDec sector generation, for example

```
.venv/bin/python FSD.py \  
--dot-file examples/graphs/triangle.dot \  
--kinematics examples/graphs/triangle_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both  
  
.venv/bin/python FSD.py \  
--dot-file examples/graphs/box.dot \  
--kinematics examples/graphs/box_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both
```

The compact multi-loop DOT smoke checks are

```
.venv/bin/python FSD.py \  
--dot-file examples/graphs/kite_2loop.dot \  
--kinematics examples/graphs/kite_2loop_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both  
  
.venv/bin/python FSD.py \  
--dot-file examples/graphs/self_energy_3loop.dot \  
--kinematics examples/graphs/self_energy_3loop_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both  
  
.venv/bin/python FSD.py \  
--dot-file examples/graphs/three_point_2loop.dot \  
--kinematics examples/graphs/three_point_2loop_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both  
  
.venv/bin/python FSD.py \  
--dot-file examples/graphs/three_point_3loop.dot \  
--kinematics examples/graphs/three_point_3loop_kinematics.yaml \  
--sector-method iterative \  
--dot-engine both
```

```
.venv/bin/python FSD.py \  
  --dot-file examples/graphs/three_point_2loop_6line.dot \  
  --kinematics examples/graphs/three_point_2loop_6line_kinematics.yaml \  
  --sector-method iterative \  
  --dot-engine both  
  
.venv/bin/python FSD.py \  
  --dot-file examples/graphs/three_point_3loop_8line.dot \  
  --kinematics examples/graphs/three_point_3loop_8line_kinematics.yaml \  
  --sector-method iterative \  
  --dot-engine both
```

Massless timelike commands, such as `--integral triangle --s 1.0 --m 0.0`, are rejected by validation. Built-in triangle and box runs use `OneLoopBridge` unless an explicit target is provided. DOT/FSD-only runs report no pull unless a target is provided or `pySecDec` is run through `--dot-engine both`. The implementation intentionally imports neither `scipy` nor `sympy`; all symbolic handling is done through Symbolica evaluators.